



BrightLearn
AI & DATA SKILLS

STUDENT EXERCISE · SQL DATE
FUNCTIONS

SQL Date Functions Exercise

Practise DAYNAME, MONTH, YEAR, DATEDIFF, DATEADD,
DATE_TRUNC, LAST_DAY and more across twenty questions.
Extract, format, calculate — write the SQL.

Name: Talifhani

Surname: Nemutudi

Date: 31 May 2026

APRIL DATA ANALYTICS INTAKE

Questions 1–5 — Extract Date Parts

Question 01

Expected columns: order_id, customer_id, order_date, day_name

SQL Query

```
SELECT order_id, customer_id, order_date, DAYNAME(order_date) AS day_name FROM orders;
```

	¹ ₂ ³ order_id	¹ ₂ ³ customer_id	 order_date	^A _B ^C day_name
1	1001	101	2026-05-01	Fri
2	1002	102	2026-05-02	Sat
3	1003	103	2026-05-03	Sun
4	1004	104	2026-05-04	Mon
5	1005	105	2026-05-05	Tue

  5 rows | 0.657s runtime [See performance \(1\)](#) ✓ Checked for improvements

Question 02

Expected columns: customer_id, customer_name, signup_date, signup_month_name

SQL Query

```
SELECT customer_id, customer_name, signup_date, MONTHNAME(signup_date) AS signup_month_name FROM customer_signups;
```

	¹ ₂ ³ customer_id	^A _B ^C customer_name	 signup_date	^A _B ^C signup_month_name
1	201	John	2026-01-15	Jan
2	202	Mary	2026-02-20	Feb
3	203	Peter	2026-03-05	Mar
4	204	Sarah	2026-04-18	Apr
5	205	Thabo	2026-05-30	May

← Results 4 of 4 →   5 rows | 15.260s runtime [See performance \(4\)](#) ✓ Checked for improvements

Question 03

Expected columns: sale_id, product_name, sale_date, sale_month

SQL Query

```
SELECT sale_id, product_name, sale_date,
       MONTH(sale_date) AS sale_month
FROM SALESQ3;
```

Table ▼ +					
	^A _C sa...	^A _C product_name	sale_date	¹ ² ₃ sale_month	
1	S001	Laptop	2026-01-10	1	
2	S002	Mouse	2026-02-15	2	
3	S003	Keyboard	2026-03-20	3	
4	S004	Monitor	2026-04-25	4	
5	S005	Desk	2026-05-30	5	

▼

5 rows 0.346s runtime See performance (1) Checked for improvements

Question 04

Expected columns: transaction_id, customer_id, transaction_date, transaction_year

SQL Query

```
SELECT transaction_id, customer_id, transaction_date,
       YEAR(transaction_date) AS transaction_year
FROM transactionsQ4;
```

Table ▼ +					
	^A _C transaction_id	¹ ² ₃ customer_id	transaction_date	¹ ² ₃ transaction_year	
1	T001	101	2024-12-15	2024	
2	T002	102	2025-01-20	2025	
3	T003	103	2025-06-10	2025	
4	T004	104	2026-02-05	2026	
5	T005	105	2026-05-25	2026	

Question 05

Expected columns: delivery_id, customer_id, delivery_date, day_of_month

SQL Query

```
SELECT delivery_id, customer_id, delivery_date,  
       DAY(delivery_date) AS day_of_month  
FROM deliveries;
```

	^A _C delivery_id	¹ ₃ customer_id	delivery_date	¹ ₃ day_of_month
1	D001	101	2026-05-01	1
2	D002	102	2026-05-08	8
3	D003	103	2026-05-15	15
4	D004	104	2026-05-22	22
5	D005	105	2026-05-29	29

5 rows | 0.281s runtime [See performance \(1\)](#) Checked for improvements

Question 06

Expected columns: employee_id, employee_name, department, today_date

SQL Query

```
SELECT employee_id, employee_name, department,  
       CURRENT_DATE() AS today_date  
FROM employees;
```

	¹ ₃ employee_id	^A _C employee_name	^A _C department	today_date
1	301	Nandi	Sales	2026-07-26
2	302	Brian	IT	2026-07-26
3	303	Lerato	Finance	2026-07-26
4	304	Sipho	HR	2026-07-26
5	305	Aisha	Marketing	2026-07-26

5 rows | 0.523s runtime [See performance \(1\)](#) Checked for improvements

Question 07

Expected columns: order_id, customer_id, order_date_text, order_date

SQL Query

```
SELECT
    order_id,
    customer_id,
    order_date_text,
    TO_DATE(order_date_text, 'yyyy-MM-dd') AS order_date
FROM ONLINE_ORDERS;
```

Table +					
	¹ ₂ 3 order_id	¹ ₂ 3 customer_id	^A _B ^C order_date_text	^A _B ^C order_date	
1	4001	101	2026-01-15	2026-01-15	
2	4002	102	2026-02-20	2026-02-20	
3	4003	103	2026-03-25	2026-03-25	
4	4004	104	2026-04-10	2026-04-10	
5	4005	105	2026-05-05	2026-05-05	

5 rows | 0.367s runtime | [See performance \(1\)](#) | ☒ Checked for improvements

Question 08

Expected columns: payment_id, customer_id, payment_date, formatted_payment_date

SQL Query

```
SELECT payment_id, customer_id, payment_date,
       TO_CHAR(payment_date, 'yyy-MM-dd') AS formatted_payment_date
FROM payment_dates;
```

Table +					
	^A _B ^C payme...	¹ ₂ 3 customer_id	^A _B ^C payment_date	^A _B ^C formatted_payment_date	
1	P001	101	2026-01-05	2026-01-05	
2	P002	102	2026-02-10	2026-02-10	
3	P003	103	2026-03-15	2026-03-15	
4	P004	104	2026-04-20	2026-04-20	
5	P005	105	2026-05-25	2026-05-25	

Question 09

Expected columns: customer_id, customer_name, last_purchase_date, days_since_last_purchase

SQL Query

```
SELECT customer_id, customer_name, last_purchase_date,  
       DATEDIFF(CURRENT_DATE(), last_purchase_date) AS days_since_last_purchase  
FROM customer_purchases;
```

Table ▼ +				
	¹ ₂ customer_id	^A _B customer_name	last_purchase_date	¹ ₂ days_since_last_purchase
1	501	John	2026-05-01	86
2	502	Mary	2026-05-10	77
3	503	Peter	2026-05-15	72
4	504	Sarah	2026-05-20	67
5	505	Thabo	2026-05-25	62

▼ 5 rows 0.306s runtime [See performance \(1\)](#) Checked for improvements

Question 10

Expected columns: Calculate the expected delivery date by adding 7 days to the order_date.

SQL Query

```
SELECT  
    order_id,  
    customer_id,  
    order_date,  
    date_add(order_date, 7) AS expected_delivery_date  
FROM SHIPPING_ORDERS;
```

Table ▼ +				
	¹ ₂ order_id	¹ ₂ customer_id	order_date	expected_delivery_date
1	6001	101	2026-05-01	2026-05-08
2	6002	102	2026-05-03	2026-05-10
3	6003	103	2026-05-05	2026-05-12
4	6004	104	2026-05-07	2026-05-14
5	6005	105	2026-05-09	2026-05-16

03 Questions 11–15 — Multi-Extract & Date Filtering

Question 11

Expected columns: booking_id, customer_id, booking_date, booking_year, booking_month, booking_day

SQL Query

```
SELECT
    booking_id,
    customer_id,
    booking_date,
    YEAR(booking_date) AS booking_year,
    MONTH(booking_date) AS booking_month,
    DAY(booking_date) AS booking_day
FROM BOOKINGS;
```

Add parameter							
Table							
	booking_id	customer_id	booking_date	booking_year	booking_month	booking_day	
1	B001	101	2026-01-12	2026	1	12	
2	B002	102	2026-02-18	2026	2	18	
3	B003	103	2026-03-22	2026	3	22	
4	B004	104	2026-04-09	2026	4	9	
5	B005	105	2026-05-27	2026	5	27	
5 rows 0.750s runtime See performance (1) ✓ Checked for improvements Refreshed Now							

Question 12

Expected columns: order_id, customer_id, order_date, order_year, amount

SQL Query

```
SELECT order_id, customer_id, order_date,
    YEAR(order_date) AS order_year, amount
FROM yearly_ordersQ12
WHERE YEAR(order_date) = 2026;
```

Table						
	order_id	customer_id	order_date	order_year	amount	
1	7004	104	2026-02-05	2026	1500.00	
2	7005	105	2026-05-25	2026	2000.00	

Question 13

Expected columns: order_id, customer_id, order_date, order_month, amount

SQL Query

```
SELECT order_id, customer_id, order_date,  
       MONTH(order_date) AS order_month, amount  
FROM monthly_orders
```

Table	+					
	1 ² ₃ order_id	1 ² ₃ customer_id	📅 order_date	1 ² ₃ order_month	.00 amount	
1	8003	103	2026-03-10	3	800.00	
2	8004	104	2026-03-25	3	1500.00	

Question 14

Expected columns: subscription_id, customer_id, start_date, month_end_date

SQL Query

```
SELECT subscription_id, customer_id, start_date,  
       LAST_DAY(start_date) AS month_end_date  
FROM subscriptions;
```

←	Results 2 of 2	→	Table	+				
	A ^B _C subscription_id	1 ² ₃ customer_id	📅 start_date	📅 month_end_date				
1	SUB001	101	2026-01-10	2026-01-31				
2	SUB002	102	2026-02-15	2026-02-28				
3	SUB003	103	2026-03-20	2026-03-31				
4	SUB004	104	2026-04-25	2026-04-30				
5	SUB005	105	2026-05-30	2026-05-31				

📄

5 rows

2.485s runtime

[📊 See performance \(2\)](#)

✓ Checked for improvements

Question 15

Expected columns: send_id, customer_id, send_date, month_start_date

SQL Query

```
SELECT
    send_id,
    customer_id,
    send_date,
    CAST(date_trunc('month', send_date) AS DATE) AS month_start_date
FROM campaign_sends;
```

Table +				
	send_id	customer_id	send_date	month_start_date
1	C001	101	2026-01-12	2026-01-01
2	C002	102	2026-02-18	2026-02-01
3	C003	103	2026-03-22	2026-03-01
4	C004	104	2026-04-09	2026-04-01
5	C005	105	2026-05-27	2026-05-01

5 rows | 0.562s runtime | See performance (1) | Checked for improvements

Questions 16–20 — Advanced Date Work

Question 16

Expected columns: invoice_id, customer_id, invoice_date, invoice_month_year

SQL Query

```
SELECT invoice_id, customer_id, invoice_date,
    date_format(invoice_date, 'MMMM yyyy') AS invoice_month_year
FROM INVOICE_DATES;
```

Results 2 of 2 Table +				
	invoice_id	customer_id	invoice_date	invoice_month_year
1	INV001	101	2026-01-05	January 2026
2	INV002	102	2026-02-10	February 2026
3	INV003	103	2026-03-15	March 2026
4	INV004	104	2026-04-20	April 2026
5	INV005	105	2026-05-25	May 2026

5 rows | 0.873s runtime | See performance (2) | Checked for improvements

Question 17

Expected columns: customer_id, customer_name, date_of_birth, customer_age

SQL Query

```
SELECT
    customer_id,
    customer_name,
    date_of_birth,
    FLOOR(months_between(current_date(), date_of_birth) / 12) AS customer_age
FROM CUSTOMER_BIRTHDAYS;
```

Table

+

	123 customer...	⋮ ⇅	A ^B C customer_name	📅 date_of_birth	123 customer_age
1		901	John	1998-05-10	28
2		902	Mary	1990-08-20	35
3		903	Peter	2002-03-15	24
4		904	Sarah	1985-12-01	40
5		905	Thabo	2000-07-30	25

↓

▼

5 rows

0.715s runtime

[📊 See performance \(1\)](#)

✓

Checked for improvements

Question 18

Expected columns: order_id, customer_id, order_date, day_name, day_type

SQL Query

```
SELECT
    order_id,
    customer_id,
    order_date,
    date_format(order_date, 'EEEE') AS day_name,
    CASE
        WHEN dayofweek(order_date) IN (1, 7)
        THEN 'Weekend'
        ELSE 'Weekday'
    END AS day_type
FROM ORDERS;
```

	^{1 2 3} order_id	^{1 2 3} customer_id	 order_date	^{A B C} day_name	^{A B C} day_type
1	1001	101	2026-05-01	Friday	Weekday
2	1002	102	2026-05-02	Saturday	Weekend
3	1003	103	2026-05-03	Sunday	Weekend
4	1004	104	2026-05-04	Monday	Weekday
5	1005	105	2026-05-05	Tuesday	Weekday

Question 19

Expected columns: transaction_id, customer_id, transaction_date, transaction_quarter, amount

SQL Query

```
SELECT transaction_id, customer_id, transaction_date,
       QUARTER(transaction_date) AS transaction_quarter, amount
FROM quarterly_transactions;
```

	^{A B C} transaction_id	^{1 2 3} customer_id	 transaction_date	^{1 2 3} transaction_quarter	.00 amount
1	Q001	101	2026-01-15	1	500.00
2	Q002	102	2026-03-20	1	1200.00
3	Q003	103	2026-04-10	2	800.00
4	Q004	104	2026-07-05	3	1500.00
5	Q005	105	2026-10-25	4	2000.00


 5 rows | 0.216s runtime | [See performance \(1\)](#) |  Checked for improvements

Question 20

Expected columns: order_id, customer_id, order_date, days_since_order, amount

SQL Query

```
SELECT order_id, customer_id, order_date,
       DATEDIFF(CURRENT_DATE(), order_date) AS days_since_order, amount
FROM recent_orders
WHERE DATEDIFF(CURRENT_DATE(), order_date) > 30;
```

	order_id	customer_id	order_date	days_since_order	amount
1	R001	101	2026-04-01	116	500.00
2	R002	102	2026-04-15	102	1200.00
3	R003	103	2026-05-01	86	800.00
4	R004	104	2026-05-10	77	1500.00
5	R005	105	2026-05-25	62	2000.00

QB Bonus – Customer Recency Segment.

Question 21

Expected columns: customer_id, customer_name, last_purchase_date, days_since_last_purchase, customer_status

```
SELECT
    customer_id,
    customer_name,
    last_purchase_date,
    datediff(current_date(), last_purchase_date) AS days_since_last_purchase,
    CASE
        WHEN datediff(current_date(), last_purchase_date) <= 30 THEN 'Active Customer'
        WHEN datediff(current_date(), last_purchase_date) BETWEEN 31 AND 90 THEN 'At Risk Customer'
        ELSE 'Inactive Customer'
    END AS customer_status
FROM CUSTOMER_PURCHASES;
```

	customer_id	customer_name	last_purchase_date	days_since_last_purchase	customer_status
1	501	John	2026-05-01	86	At Risk Customer
2	502	Mary	2026-05-10	77	At Risk Customer
3	503	Peter	2026-05-15	72	At Risk Customer
4	504	Sarah	2026-05-20	67	At Risk Customer
5	505	Thabo	2026-05-25	62	At Risk Customer

5 rows 0.776s runtime See performance (1) ✓ Checked for improvements Refreshed Now